

React API Fetching Notes (Fetch & Axios)

What is an API?

API (Application Programming Interface) is a way for two applications to communicate.

React Application $\rightleftarrows$ API $\rightleftarrows$ Database

Example:

- Weather API
 - GitHub API
 - MockAPI
 - JSONPlaceholder
-

What is Fetch API?

The Fetch API is a built-in JavaScript function used to make HTTP requests.

No installation required.

```
fetch(url)
```

What is Axios?

Axios is a popular JavaScript library for making HTTP requests.

Installation

```
npm install axios
```

Import it

```
import axios from "axios";
```

HTTP Methods

Method	Purpose
GET	Read Data
POST	Create Data
PUT	Update Entire Data
PATCH	Update Partial Data
DELETE	Delete Data

Mock API

You can create a fake REST API using:

- MockAPI
- JSON Server
- JSONPlaceholder

Example endpoint

```
https://jsonplaceholder.typicode.com/users
```

or

```
https://66b12345.mockapi.io/users
```

React useEffect

API calls should usually be made inside `useEffect`.

```
import { useEffect } from "react";

useEffect(() => {
  // API Call
}, []);
```

`[]` means the API runs only once when the component mounts.

GET Request using Fetch

```
import { useEffect, useState } from "react";

function App() {

  const [users, setUsers] = useState([]);

  useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
      });

  }, []);

  return (
    <div>

      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}

    </div>
  )
}
```

GET Request using Async/Await

```
import { useEffect, useState } from "react";

function App() {

  const [users, setUsers] = useState([]);

  useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response)=>response.json())
      .then((data)=>{
        setUsers(data);
      });

  }, []);

  return (
    <div>

      {users.map(user=>(
        <p key={user.id}>{user.name}</p>
      ))}

    </div>
  )
}
```

GET Request using Axios

```
import axios from "axios";

const getUsers = async () => {

  const response = await axios.get(
    "https://jsonplaceholder.typicode.com/users"
  );

  console.log(response.data);
}
```

```
};
```

Why Axios?

Advantages

- Cleaner syntax
- Automatic JSON conversion
- Better error handling
- Request timeout
- Interceptors
- Supports older browsers

POST Request (Fetch)

```
const addUser = async () => {  
  
  const response = await fetch(  
    "https://jsonplaceholder.typicode.com/users",  
    {  
      method: "POST",  
  
      headers: {  
        "Content-Type": "application/json"  
      },  
  
      body: JSON.stringify({  
  
        name: "Ali",  
        email: "ali@gmail.com"  
      })  
    }  
  );  
  
  const data = await response.json();
```

```
console.log(data);  
  
}
```

POST Request (Axios)

```
import axios from "axios";  
  
const addUser = async () => {  
  
  const response = await axios.post(  
    "https://jsonplaceholder.typicode.com/users",  
    {  
      name: "Ali",  
      email: "ali@gmail.com"  
    }  
  );  
  
  console.log(response.data);  
  
}
```

PUT Request (Fetch)

```
const updateUser = async () => {  
  
  await fetch(  
    "https://jsonplaceholder.typicode.com/users/1",  
    {  
  
      method: "PUT",  
  
      headers: {  
        "Content-Type": "application/json"  
      },  
  
      body: JSON.stringify({  
  
        name: "Ahmed",  
        email: "ahmed@gmail.com"  
      })  
    }  
  );  
  
}
```

```
})  
  
}  
  
);  
  
};
```

PUT Request (Axios)

```
await axios.put(  
  "https://jsonplaceholder.typicode.com/users/1",  
  {  
    name: "Ahmed",  
    email: "ahmed@gmail.com"  
  }  
);
```

DELETE Request (Fetch)

```
await fetch("https://jsonplaceholder.typicode.com/users/1",  
  {  
    method: "DELETE"  
  }  
);
```

DELETE Request (Axios)

```
await axios.delete("https://jsonplaceholder.typicode.com/users/1"  
);
```

Complete CRUD Flow

Create → POST

Read → GET

Update → PUT

Delete → DELETE

Error Handling (Fetch)

```
try{

  const response = await fetch(url);

  if(!response.ok){
    throw new Error("Something went wrong");
  }

  const data = await response.json();

}catch(error){

  console.log(error);

}
```

Error Handling (Axios)

```
try{

  const response = await fetch(url);

  if(!response.ok){
    throw new Error("Something went wrong");
  }

  const data = await response.json();

}catch(error){
```

```
console.log(error);  
  
}
```

Loading State

```
const [loading, setLoading]=useState(false);  
  
const getUsers=async()=>{  
  
  setLoading(true);  
  
  try{  
  
    const response=await axios.get(url);  
  
    setUsers(response.data);  
  
  }  
  finally{  
  
    setLoading(false);  
  
  }  
  
}
```

Display Loading

```
{  
  loading ? (  
    <p>Loading...</p>  
  ) : (  
    users.map(user=>(  
      <p key={user.id}>{user.name}</p>  
    ))  
  )  
}
```

Rendering API Data

```
{users.map((user)=>(  
<div key={user.id}>  
<h3>{user.name}</h3>  
<p>{user.email}</p>  
</div>  
))}
```

useEffect with Axios

```
useEffect(()=>{getUsers();  
  
},[]);
```

Fetch vs Axios

Feature	Fetch	Axios
Installation	✗ No	✓ Yes
JSON Conversion	Manual (<code>response.json()</code>)	Automatic
Error Handling	Manual	Better
Timeout	Manual	Built-in
Request Cancel	Limited	Easy
Interceptors	✗	✓
File Upload	More setup	Easier

Folder Structure

```
src  
|— components
```

```
|— services
|   api.js

|— App.js
```

Service File (Axios)

```
import axios from "axios";

const API = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com"
});

export default API;
```

Usage

```
import axios from "axios";

const API = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com"
});

export default API;
```

Best Practices

- Use Axios for medium and large projects.
- Keep API calls in a separate `services` folder.
- Use `async/await` instead of chained `.then()`.
- Handle loading and error states.
- Avoid making API requests directly inside JSX.
- Store fetched data in React state using `useState`.

- Trigger API calls inside `useEffect` when the component loads.
- Use environment variables for API URLs and keys instead of hardcoding them.